File: backend_fastapi_mastery/00_http_and_rest_foundations.md

HTTP and REST Foundations for Backend Engineers

Why This Matters

Every backend interview, every API design review, every production incident involving your API will test your understanding of HTTP fundamentals. This isn't academic knowledge—it's the foundation that separates junior developers from senior engineers who can reason about distributed systems.

When a senior engineer asks "Is this endpoint idempotent?" or "Why did you choose PUT over PATCH?", they're testing whether you understand the contract you're creating with your API consumers.

HTTP: The Protocol That Powers the Web

The Request-Response Model

HTTP is a stateless, request-response protocol. Every interaction follows this pattern:

```
Client sends REQUEST → Server processes → Server sends RESPONSE
```

Stateless means the server doesn't inherently remember previous requests. Each request must contain all information needed to process it. This has profound implications:

1. **Scalability**: Any server can handle any request (horizontal scaling)
2. **Simplicity**: No server-side session state to manage
3. **Trade-off**: Client must send authentication/context with every request

Anatomy of an HTTP Request

```
POST /api/v1/users HTTP/1.1
Host: api.example.com
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
X-Request-ID: 550e8400-e29b-41d4-a716-446655440000

{
    "email": "user@example.com",
    "name": "John Doe"
}
```

Line-by-line breakdown:

Component	Example	Purpose
Method	`POST`	What action to perform
Path	`/api/v1/users`	Which resource
Protocol	`HTTP/1.1`	Protocol version
Host Header	`api.example.com`	Target server (required in HTTP/1.1)
Content-Type	`application/json`	Format of request body
Authorization	`Bearer ...`	Authentication credentials
X-Request-ID	UUID	Correlation ID for tracing
Body	JSON	The actual payload

Anatomy of an HTTP Response

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/v1/users/12345
X-Request-ID: 550e8400-e29b-41d4-a716-446655440000

{
    "id": "12345",
    "email": "user@example.com",
    "name": "John Doe",
    "created_at": "2024-01-15T10:30:00Z"
}
```

Key response elements:

Component	Example	Purpose
Status Code	`201`	Outcome category
Status Text	`Created`	Human-readable status
Location	URL	Where the new resource lives
Body	JSON	Response data

HTTP Methods: Beyond GET and POST

The Complete Method Semantics

Method	Safe?	Idempotent?	Has Body?	Typical Use
GET	Yes	Yes	No	Retrieve resource

Method	Safe?	Idempotent?	Has Body?	Typical Use
HEAD	Yes	Yes	No	Check resource exists
POST	No	No	Yes	Create resource, trigger action
PUT	No	Yes	Yes	Replace entire resource
PATCH	No	No*	Yes	Partial update
DELETE	No	Yes	Optional	Remove resource
OPTIONS	Yes	Yes	No	CORS preflight, capability discovery

Safe vs Unsafe Methods

Safe methods don't modify server state. Calling them has no side effects (beyond logging/analytics).

```
# SAFE: GET request - server state unchanged
@app.get("/users/{user_id}")
async def get_user(user_id: int):
    # Only reads data, never modifies
    return await user_repository.find_by_id(user_id)
```

Why this matters in production:

1. Safe methods can be cached aggressively
2. Safe methods can be retried freely by proxies/browsers
3. Crawlers will follow safe method links
4. Load balancers can redirect safe requests anywhere

Unsafe methods modify server state:

```
# UNSAFE: POST creates new state
@app.post("/users")
async def create_user(user: UserCreate):
    # Creates new data - state changes
    return await user_repository.create(user)
```

Idempotency: The Critical Concept

Idempotent operation: Calling it once has the same effect as calling it multiple times.

This is **critical** in distributed systems because:

- Networks fail
- Clients retry
- Timeouts happen
- Load balancers retry

```python
# IDEMPOTENT: PUT with specific ID
# Calling this 1 time or 100 times = same final state
@app.put("/users/{user_id}")
async def replace_user(user_id: int, user: UserUpdate):
    """
    Replace user entirely. If user exists, overwrite.
    Calling multiple times with same data = same result.
    """
    return await user_repository.upsert(user_id, user)

# NOT IDEMPOTENT: POST creates new resource each time
# Calling this 3 times = 3 new users
@app.post("/users")
async def create_user(user: UserCreate):
    """
    Each call creates a new user with new ID.
    3 calls = 3 users.
    """
    return await user_repository.create(user)
```

The PUT vs POST vs PATCH Decision

Use POST when:

- Client doesn't know the resource ID
- Action is not idempotent
- You're triggering a process, not creating a "thing"

```python
# POST: Server generates ID
@app.post("/orders")
async def create_order(order: OrderCreate):
    order_id = generate_uuid()  # Server decides ID
    return await order_repository.create(order_id, order)

# POST: Triggering an action (not creating a resource)
@app.post("/users/{user_id}/send-verification-email")
async def send_verification(user_id: int):
    # This triggers a side effect, not creating a resource
    await email_service.send_verification(user_id)
    return {"status": "sent"}
```

Use PUT when:

- Client knows/controls the resource ID
- You're replacing the entire resource
- Operation should be idempotent

```python
# PUT: Client specifies ID, full replacement
@app.put("/documents/{document_id}")
async def replace_document(document_id: str, doc: Document):
    """
    Idempotent: same input always produces same state.
    Replaces ALL fields - missing fields become null/default.
    """
    return await document_repository.upsert(document_id, doc)
```

Use PATCH when:

- Partial update of existing resource
- Only sending fields that should change

```python
# PATCH: Partial update
@app.patch("/users/{user_id}")
async def update_user(user_id: int, updates: UserPatch):
    """
    Only updates provided fields.
    Other fields remain unchanged.
    """
    existing = await user_repository.find_by_id(user_id)
    if not existing:
        raise HTTPException(status_code=404)

    update_data = updates.dict(exclude_unset=True)
    return await user_repository.update(user_id, update_data)
```

Interview insight: PATCH idempotency is nuanced. `PATCH /counter {"increment": 1}` is NOT idempotent. `PATCH /user {"name": "John"}` IS idempotent. The RFC says PATCH *can* be idempotent but isn't required to be.

HTTP Status Codes: Communicating Outcomes

The Five Categories

Range	Category	Meaning
1xx	Informational	Request received, continuing
2xx	Success	Request succeeded
3xx	Redirection	Further action needed
4xx	Client Error	Client made a mistake
5xx	Server Error	Server failed

Status Codes You Must Know

Success Codes (2xx)

```python
# 200 OK - Generic success, has response body
@app.get("/users/{user_id}")
async def get_user(user_id: int):
    user = await user_repository.find_by_id(user_id)
    return user  # FastAPI returns 200 by default

# 201 Created - New resource created
@app.post("/users", status_code=201)
async def create_user(user: UserCreate, response: Response):
    created = await user_repository.create(user)
    response.headers["Location"] = f"/users/{created.id}"
    return created

# 202 Accepted - Request accepted, processing async
@app.post("/reports", status_code=202)
async def generate_report(request: ReportRequest):
    """
    Report generation takes minutes.
    We accept the request and process in background.
    """
    job_id = await job_queue.enqueue(generate_report_task, request)
    return {"job_id": job_id, "status_url": f"/jobs/{job_id}"}

# 204 No Content - Success, no body to return
@app.delete("/users/{user_id}", status_code=204)
async def delete_user(user_id: int):
    await user_repository.delete(user_id)
    return None  # No response body
```

Client Error Codes (4xx)

```python
from fastapi import HTTPException, status

# 400 Bad Request - Malformed request syntax
@app.post("/users")
async def create_user(user: UserCreate):
    if not is_valid_email(user.email):
        raise HTTPException(
            status_code=400,
            detail="Invalid email format"
        )

# 401 Unauthorized - Authentication required/failed
@app.get("/me")
async def get_current_user(token: str = Depends(oauth2_scheme)):
    user = await verify_token(token)
    if not user:
        raise HTTPException(
```

```
            status_code=401,
            detail="Invalid or expired token",
            headers={"WWW-Authenticate": "Bearer"}
        )
    return user

# 403 Forbidden - Authenticated but not authorized
@app.delete("/admin/users/{user_id}")
async def admin_delete_user(
    user_id: int,
    current_user: User = Depends(get_current_user)
):
    if not current_user.is_admin:
        raise HTTPException(
            status_code=403,
            detail="Admin access required"
        )

# 404 Not Found - Resource doesn't exist
@app.get("/users/{user_id}")
async def get_user(user_id: int):
    user = await user_repository.find_by_id(user_id)
    if not user:
        raise HTTPException(status_code=404, detail="User not found")
    return user

# 409 Conflict - Request conflicts with current state
@app.post("/users")
async def create_user(user: UserCreate):
    existing = await user_repository.find_by_email(user.email)
    if existing:
        raise HTTPException(
            status_code=409,
            detail="User with this email already exists"
        )

# 422 Unprocessable Entity - Validation failed (FastAPI default)
# FastAPI automatically returns 422 when Pydantic validation fails

# 429 Too Many Requests - Rate limit exceeded
@app.get("/api/data")
async def get_data(request: Request):
    if await rate_limiter.is_exceeded(request.client.host):
        raise HTTPException(
            status_code=429,
            detail="Rate limit exceeded",
            headers={"Retry-After": "60"}
        )
```

Server Error Codes (5xx)

```python
# 500 Internal Server Error - Unexpected server failure
# Usually you don't explicitly raise this - it happens on unhandled
exceptions

# 502 Bad Gateway - Upstream service failed
@app.get("/external-data")
async def get_external_data():
    try:
        return await external_api.fetch()
    except ExternalAPIError:
        raise HTTPException(
            status_code=502,
            detail="External service unavailable"
        )

# 503 Service Unavailable - Server temporarily overloaded
@app.get("/health")
async def health_check():
    if not await database.is_healthy():
        raise HTTPException(
            status_code=503,
            detail="Database unavailable",
            headers={"Retry-After": "30"}
        )
    return {"status": "healthy"}

# 504 Gateway Timeout - Upstream timed out
@app.get("/slow-operation")
async def slow_operation():
    try:
        return await asyncio.wait_for(
            external_service.call(),
            timeout=30.0
        )
    except asyncio.TimeoutError:
        raise HTTPException(
            status_code=504,
            detail="Operation timed out"
        )
```

Status Code Decision Tree

```
Is the request successful?
├── Yes
│   ├── Creating new resource? → 201 Created
│   ├── Async processing? → 202 Accepted
│   ├── No content to return? → 204 No Content
│   └── Returning data? → 200 OK
│
└── No
```

```
    ├── Is it the client's fault?
    │   ├── Malformed request? → 400 Bad Request
    │   ├── Not authenticated? → 401 Unauthorized
    │   ├── Not authorized? → 403 Forbidden
    │   ├── Resource not found? → 404 Not Found
    │   ├── State conflict? → 409 Conflict
    │   ├── Validation failed? → 422 Unprocessable Entity
    │   └── Too many requests? → 429 Too Many Requests
    │
    └── Is it the server's fault?
        ├── Unexpected error? → 500 Internal Server Error
        ├── Upstream failed? → 502 Bad Gateway
        ├── Overloaded? → 503 Service Unavailable
        └── Upstream timeout? → 504 Gateway Timeout
```

REST: Architectural Constraints

REST (Representational State Transfer) is an architectural style, not a protocol. Roy Fielding defined six constraints:

1. Client-Server

Separation of concerns. Client handles UI, server handles data/logic.

Why it matters: Teams can evolve independently. Mobile app, web app, and CLI can all use the same API.

2. Stateless

Each request contains all information needed. Server doesn't store client session.

```python
# WRONG: Relying on server-side session
@app.post("/login")
async def login(credentials: Credentials):
    session["user_id"] = user.id  # Server stores state
    return {"status": "logged in"}

@app.get("/profile")
async def profile():
    user_id = session["user_id"]  # Depends on previous request
    return await get_user(user_id)

# RIGHT: Stateless with token
@app.post("/login")
async def login(credentials: Credentials):
    user = await authenticate(credentials)
    token = create_jwt(user.id)  # All state in token
    return {"access_token": token}

@app.get("/profile")
async def profile(token: str = Depends(oauth2_scheme)):
```

```
    user_id = decode_jwt(token)  # State comes from request
    return await get_user(user_id)
```

3. Cacheable

Responses must define themselves as cacheable or not.

```
from fastapi import Response

@app.get("/products/{product_id}")
async def get_product(product_id: int, response: Response):
    product = await product_repository.find(product_id)

    # Cacheable for 1 hour
    response.headers["Cache-Control"] = "public, max-age=3600"
    response.headers["ETag"] = f'"{hash(product)}"'

    return product

@app.get("/me")
async def get_current_user(response: Response, user = Depends(get_user)):
    # Never cache user-specific data in shared caches
    response.headers["Cache-Control"] = "private, no-store"
    return user
```

4. Uniform Interface

Consistent interface across resources. This includes:

Resource identification via URIs:

```
/users/123           # User resource
/users/123/orders    # User's orders
/orders/456          # Specific order
```

Resource manipulation through representations:

```
# Client sends representation to modify resource
@app.put("/users/{user_id}")
async def update_user(user_id: int, user: UserUpdate):
    # user is a representation of the desired state
    return await user_repository.update(user_id, user)
```

Self-descriptive messages:

```
# Response tells client how to interpret it
return Response(
    content=json.dumps(data),
    media_type="application/json",
    headers={"X-Total-Count": str(total)}
)
```

HATEOAS (Hypermedia as the Engine of Application State):

```
# Response includes links to related actions/resources
@app.get("/orders/{order_id}")
async def get_order(order_id: int):
    order = await order_repository.find(order_id)
    return {
        "id": order.id,
        "status": order.status,
        "total": order.total,
        "_links": {
            "self": f"/orders/{order.id}",
            "cancel": f"/orders/{order.id}/cancel" if order.can_cancel
else None,
            "items": f"/orders/{order.id}/items",
            "customer": f"/users/{order.customer_id}"
        }
    }
```

5. Layered System

Client can't tell if connected directly to server or through intermediary (load balancer, cache, etc.).

6. Code on Demand (Optional)

Server can extend client by sending executable code (JavaScript). Rarely used in APIs.

REST API Design Best Practices

URL Design

Use nouns, not verbs:

```
# WRONG
POST /createUser
GET /getUser/123
POST /deleteUser/123

# RIGHT
POST /users
```

```
GET /users/123
DELETE /users/123
```

Use plural nouns:

```
# Consistent: always plural
GET /users          # List users
GET /users/123      # Get user
POST /users         # Create user
GET /orders/456/items   # Items in order
```

Nest for relationships (but don't go too deep):

```
# Good: clear relationship
GET /users/123/orders
GET /orders/456/items

# Too deep: hard to understand and implement
GET /users/123/orders/456/items/789/reviews

# Better: flatten when relationship is clear
GET /order-items/789/reviews
```

Use query parameters for filtering/sorting:

```
GET /users?status=active&sort=-created_at&limit=20&offset=40
```

Resource Representation

Consistent response structure:

```
# Single resource
{
    "id": "123",
    "email": "user@example.com",
    "name": "John Doe",
    "created_at": "2024-01-15T10:30:00Z",
    "updated_at": "2024-01-15T10:30:00Z"
}

# Collection with pagination
{
    "data": [...],
    "pagination": {
        "total": 100,
```

```
        "limit": 20,
        "offset": 0,
        "has_more": true
    }
}

# Error response
{
    "error": {
        "code": "VALIDATION_ERROR",
        "message": "Invalid input",
        "details": [
            {"field": "email", "message": "Invalid email format"}
        ]
    }
}
```

API Versioning

```
# URL versioning (most common)
@app.get("/api/v1/users")
@app.get("/api/v2/users")

# Header versioning
@app.get("/users")
async def get_users(request: Request):
    version = request.headers.get("API-Version", "1")
    if version == "2":
        return await get_users_v2()
    return await get_users_v1()
```

Production reality: URL versioning is simpler to debug, cache, and document. Header versioning is "purer" but harder operationally.

JSON Over HTTP

Content Negotiation

```
from fastapi import Request
from fastapi.responses import JSONResponse, Response

@app.get("/data")
async def get_data(request: Request):
    accept = request.headers.get("Accept", "application/json")

    data = await fetch_data()

    if "application/json" in accept:
```

```
        return JSONResponse(content=data)
    elif "text/csv" in accept:
        csv_content = convert_to_csv(data)
        return Response(content=csv_content, media_type="text/csv")
    else:
        # Default to JSON
        return JSONResponse(content=data)
```

Date/Time Handling

Always use ISO 8601 format with timezone:

```
from datetime import datetime, timezone

# WRONG: Ambiguous formats
"01/15/2024"           # US format? European?
"2024-01-15 10:30:00"  # What timezone?

# RIGHT: ISO 8601 with timezone
"2024-01-15T10:30:00Z"         # UTC
"2024-01-15T10:30:00+05:30"    # With offset
```

```
from pydantic import BaseModel
from datetime import datetime

class Event(BaseModel):
    name: str
    starts_at: datetime  # Pydantic handles ISO 8601 parsing

    class Config:
        json_encoders = {
            datetime: lambda v: v.isoformat()
        }
```

Handling Large Numbers

JavaScript number precision issue:

```
# PROBLEM: JavaScript loses precision for integers > 2^53
# Python: 9007199254740993
# JavaScript: 9007199254740992 (wrong!)

# SOLUTION: Return large IDs as strings
class Order(BaseModel):
    id: str  # "9007199254740993" - string, not int
    amount_cents: int  # Small enough for JS
```

Common Interview Questions and Answers

Q1: "What's the difference between PUT and PATCH?"

Answer: "PUT replaces the entire resource - you send the complete new state, and any fields you don't include become null or default. PATCH is a partial update - you only send the fields you want to change, and other fields remain untouched. PUT is always idempotent; PATCH can be idempotent but isn't required to be. In practice, I use PATCH more often because it's more bandwidth-efficient and less error-prone when multiple clients might update different fields."

Q2: "What status code would you return for a validation error?"

Answer: "422 Unprocessable Entity. The request syntax is correct (not 400), but the semantic content is invalid. FastAPI uses 422 by default for Pydantic validation failures. Some APIs use 400 for all client errors, which is acceptable but less precise. I prefer 422 because it clearly distinguishes 'I couldn't parse your request' (400) from 'I parsed it but the values are invalid' (422)."

Q3: "When would you use 202 Accepted?"

Answer: "When the request is valid and accepted, but processing will happen asynchronously. For example, generating a large report, processing a video upload, or any operation that takes longer than a reasonable HTTP timeout. I return 202 with a job ID and status URL so the client can poll for completion. This prevents timeout issues and gives better UX than making clients wait."

Q4: "Explain idempotency and why it matters."

Answer: "An idempotent operation produces the same result whether you call it once or multiple times. In distributed systems, this is critical because networks are unreliable - requests can timeout, be duplicated by retries, or be replayed by load balancers. If my 'create payment' endpoint isn't idempotent, a retry could charge the customer twice. I make operations idempotent using idempotency keys - the client sends a unique key, and if I see the same key again, I return the original result instead of processing again."

Q5: "Why is REST stateless? What's the tradeoff?"

Answer: "Statelessness means each request contains all information needed to process it - the server doesn't store session state between requests. This enables horizontal scaling because any server can handle any request. The tradeoff is that every request must include authentication credentials, which adds overhead. We mitigate this with lightweight tokens like JWTs. The benefits - scalability, simplicity, reliability - far outweigh the small overhead in most cases."

Anti-Patterns to Avoid

1. Verbs in URLs

```
# WRONG
POST /createUser
GET /getUserById/123
```

```
POST /user/123/delete

# RIGHT
POST /users
GET /users/123
DELETE /users/123
```

2. Ignoring HTTP Methods

```
# WRONG: Using POST for everything
POST /api?action=getUser&id=123
POST /api?action=deleteUser&id=123

# RIGHT: Semantic methods
GET /users/123
DELETE /users/123
```

3. Using 200 for Errors

```
# WRONG
@app.get("/users/{user_id}")
async def get_user(user_id: int):
    user = await find_user(user_id)
    if not user:
        return {"success": False, "error": "Not found"}  # Still 200!
    return {"success": True, "data": user}

# RIGHT
@app.get("/users/{user_id}")
async def get_user(user_id: int):
    user = await find_user(user_id)
    if not user:
        raise HTTPException(status_code=404, detail="User not found")
    return user
```

4. Deeply Nested URLs

```
# WRONG: Too deep, hard to understand
GET /companies/123/departments/456/teams/789/members/012/tasks

# RIGHT: Flatten when practical
GET /tasks?team_id=789
GET /team-members/012/tasks
```

5. Inconsistent Naming

```
# WRONG: Mixed conventions
GET /users
GET /getProducts
GET /order-items
GET /CustomerAddresses

# RIGHT: Consistent kebab-case or snake_case
GET /users
GET /products
GET /order-items
GET /customer-addresses
```

Production Considerations

Request ID Tracing

```python
import uuid
from fastapi import Request

@app.middleware("http")
async def add_request_id(request: Request, call_next):
    request_id = request.headers.get("X-Request-ID", str(uuid.uuid4()))
    # Store in request state for logging
    request.state.request_id = request_id

    response = await call_next(request)
    response.headers["X-Request-ID"] = request_id
    return response
```

Rate Limiting Headers

```
response.headers["X-RateLimit-Limit"] = "100"
response.headers["X-RateLimit-Remaining"] = "95"
response.headers["X-RateLimit-Reset"] = "1704067200"  # Unix timestamp
```

Pagination

```python
@app.get("/users")
async def list_users(
    limit: int = Query(default=20, le=100),
    offset: int = Query(default=0, ge=0),
    response: Response = None
):
    users, total = await user_repository.list(limit=limit, offset=offset)
```

```
    response.headers["X-Total-Count"] = str(total)
    response.headers["Link"] = build_pagination_links(limit, offset,
total)

    return {
        "data": users,
        "pagination": {
            "total": total,
            "limit": limit,
            "offset": offset,
            "has_more": offset + limit < total
        }
    }
```

Mastery Checkpoints

Test your understanding with these questions:

Conceptual Questions

1. **Why is GET considered safe but DELETE is not, even though DELETE is idempotent?**

   Answer: Safety and idempotency are different properties. Safe means "no side effects" - GET doesn't change server state. DELETE changes state (removes resource) but is idempotent because deleting something twice has the same effect as deleting once (it's gone). Calling DELETE /users/123 ten times still results in user 123 being deleted once.

2. **A client sends a POST request that times out. The server actually processed it. The client retries. What happens and how do you prevent double-processing?**

   Answer: Without protection, you get duplicate records/actions. Prevent with idempotency keys: client generates unique key, sends with request. Server checks if key was seen before - if yes, return cached response. If no, process and store key+response. This makes POST effectively idempotent for that specific request.

3. **When should you use 401 vs 403?**

   Answer: 401 = "I don't know who you are" (authentication failed/missing). 403 = "I know who you are, but you can't do this" (authorization failed). 401 should include WWW-Authenticate header. Example: Anonymous user accessing /admin → 401. Authenticated non-admin accessing /admin → 403.

4. **Why might you return 202 instead of 201 for a POST request?**

   Answer: When the resource isn't immediately created because processing is async. Example: POST /video-transcodes doesn't create the transcode immediately - it queues the job. Return 202 with job ID and status endpoint. The resource (transcoded video) will exist later. 201 means "resource exists now at this URL."

5. **What's the difference between 400, 422, and 500?**

Answer: 400 = malformed request (bad JSON syntax, missing required header). 422 = well-formed but semantically invalid (email field contains "notanemail"). 500 = server bug (unhandled exception, database crash). 400/422 = client should fix request. 500 = server needs to fix code.

Scenario Questions

6. **Design the endpoints for a blog system with posts, comments, and likes.**

   Answer:

```
POST   /posts              - Create post
GET    /posts              - List posts
GET    /posts/{id}         - Get post
PUT    /posts/{id}         - Replace post
PATCH  /posts/{id}         - Update post
DELETE /posts/{id}         - Delete post

GET    /posts/{id}/comments     - List comments
POST   /posts/{id}/comments     - Create comment
DELETE /comments/{id}           - Delete comment

POST   /posts/{id}/likes        - Like post
DELETE /posts/{id}/likes        - Unlike post
GET    /posts/{id}/likes/count  - Get like count
```

7. **A mobile app needs to create an order, but the network is unreliable. How do you ensure orders aren't duplicated?**

   Answer: Implement idempotency keys. Client generates UUID before request, sends as X-Idempotency-Key header. Server stores key → response mapping (with TTL). On duplicate key, return stored response. Also: use optimistic locking, database unique constraints on business keys where applicable.

8. **Your API needs to support both JSON and XML responses. How?**

   Answer: Content negotiation via Accept header. Check Accept header, return appropriate Content-Type. FastAPI example:

```python
@app.get("/data")
async def get_data(request: Request):
    if "application/xml" in request.headers.get("Accept", ""):
        return Response(content=to_xml(data),
media_type="application/xml")
    return JSONResponse(content=data)
```

9. **You're building an endpoint that deletes a user and all their data. This takes 30 seconds. How do you design it?**

Answer: Don't make client wait 30 seconds. Return 202 Accepted with job ID. Process deletion async (background task/queue). Provide status endpoint: GET /deletion-jobs/{id}. Optionally support webhook callback when complete. Consider: soft delete first (immediate), hard delete async.

10. **Your GET /users endpoint is slow. How might HTTP help?**

    Answer: Caching. Return Cache-Control headers (max-age, stale-while-revalidate). Use ETag for conditional requests - client sends If-None-Match, server returns 304 Not Modified if unchanged (saves bandwidth). Consider pagination to reduce payload size. CDN can cache responses at edge.

Code Review Questions

11. **What's wrong with this code?**

```
@app.post("/users/{user_id}")
async def create_user(user_id: int, user: UserCreate):
    return await user_repository.create(user_id, user)
```

    Answer: POST with ID in path is unusual. If client specifies ID, use PUT (idempotent resource creation at known URI). If server generates ID, use POST /users without ID in path. Current design is confusing - POST suggests server-generated ID, but path suggests client-specified ID.

12. **What's wrong with this code?**

```
@app.get("/users")
async def get_users():
    users = await user_repository.get_all()
    if not users:
        raise HTTPException(status_code=404)
    return users
```

    Answer: Empty collection isn't "not found" - it's a valid state. Return 200 with empty array `[]`. 404 is for when the resource itself doesn't exist, not when a collection is empty. The endpoint `/users` exists; it just happens to contain no users.

Interview Framing

When discussing HTTP/REST in interviews:

1. **Show depth**: Don't just say "GET retrieves data." Explain safe vs unsafe, caching implications, idempotency.

2. **Mention trade-offs**: "We could use PUT for simplicity, but PATCH is more bandwidth-efficient for mobile clients with spotty connections."

3. **Connect to production**: "I always include X-Request-ID for distributed tracing. When something fails at 3 AM, that's how we correlate logs across services."

4. **Acknowledge nuance**: "REST isn't a strict standard - there are many valid interpretations. I prefer pragmatic REST that prioritizes clear contracts and good DX over philosophical purity."

5. **Think about failure**: "Networks are unreliable. I design every mutating endpoint with idempotency in mind. What happens if the client retries?"

This foundational knowledge applies whether you're using FastAPI, Django, Express, or any other framework. The protocol is the contract between you and every client that will ever call your API.